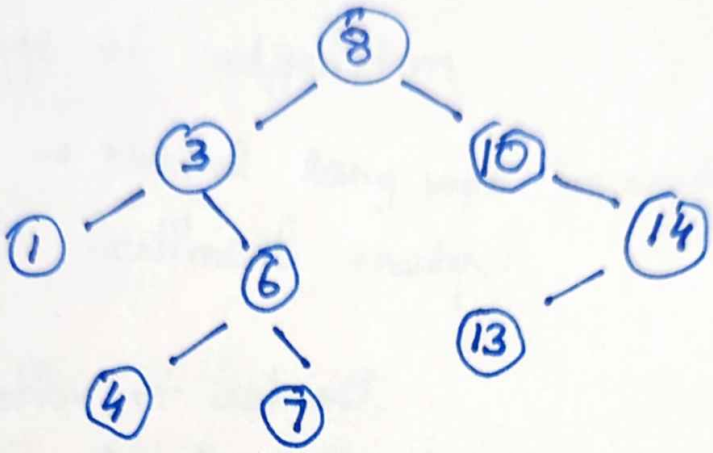
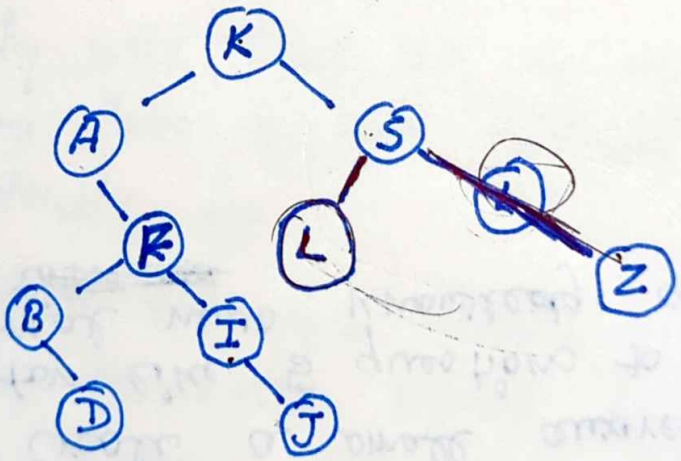
Q1

a)



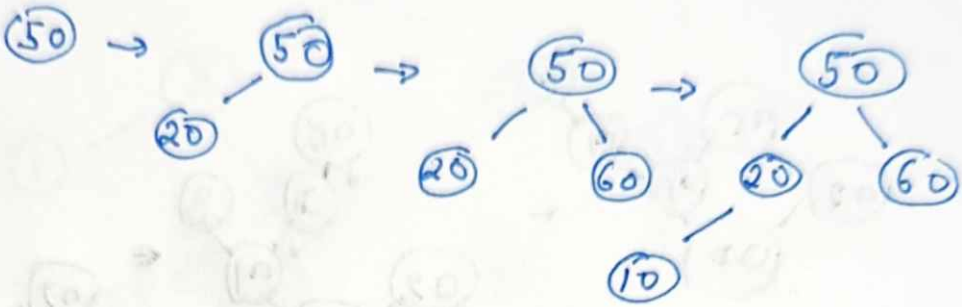
b

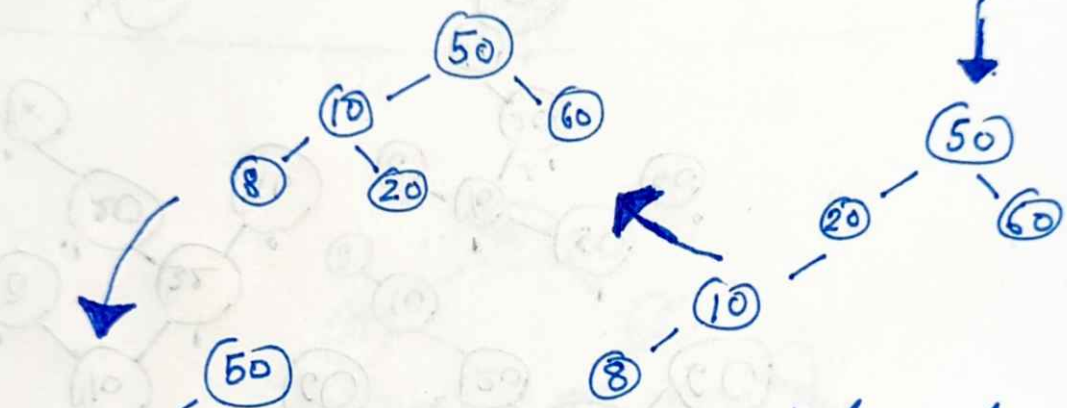


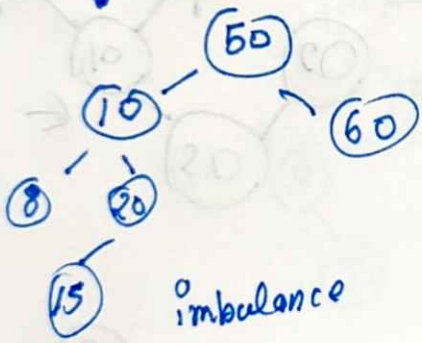
→

Q2

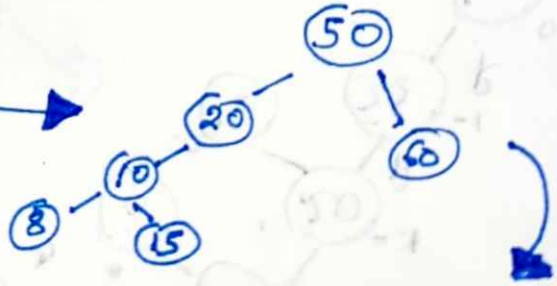
(a) 50, 20, 60, 10, 8, 15, 32, 46, 11, 48

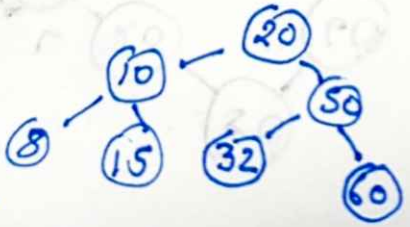


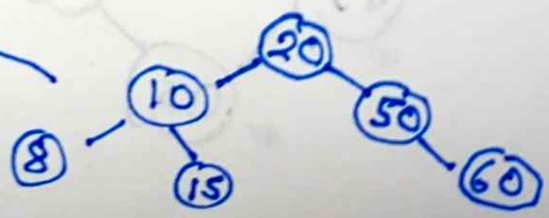


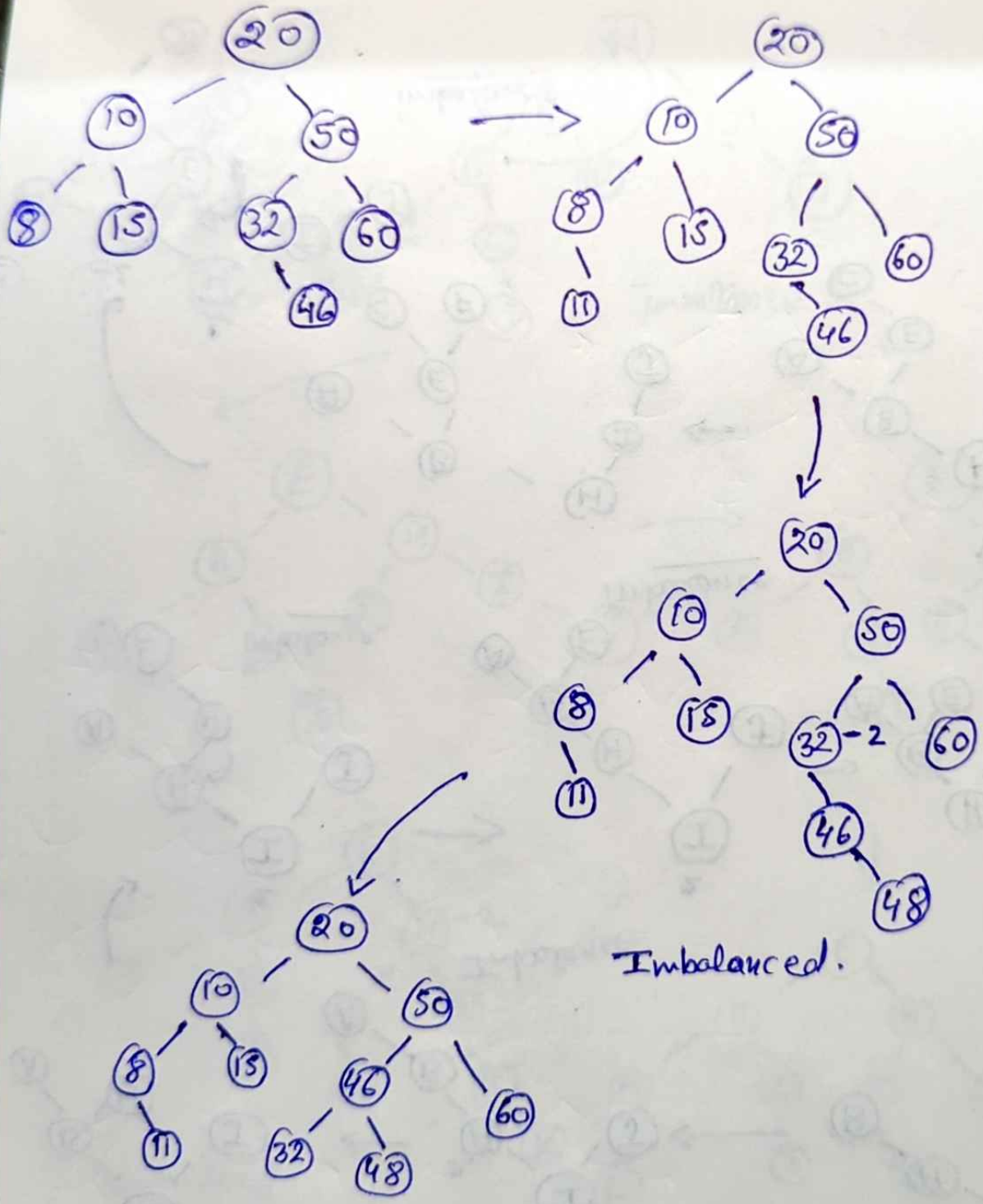


imbalance

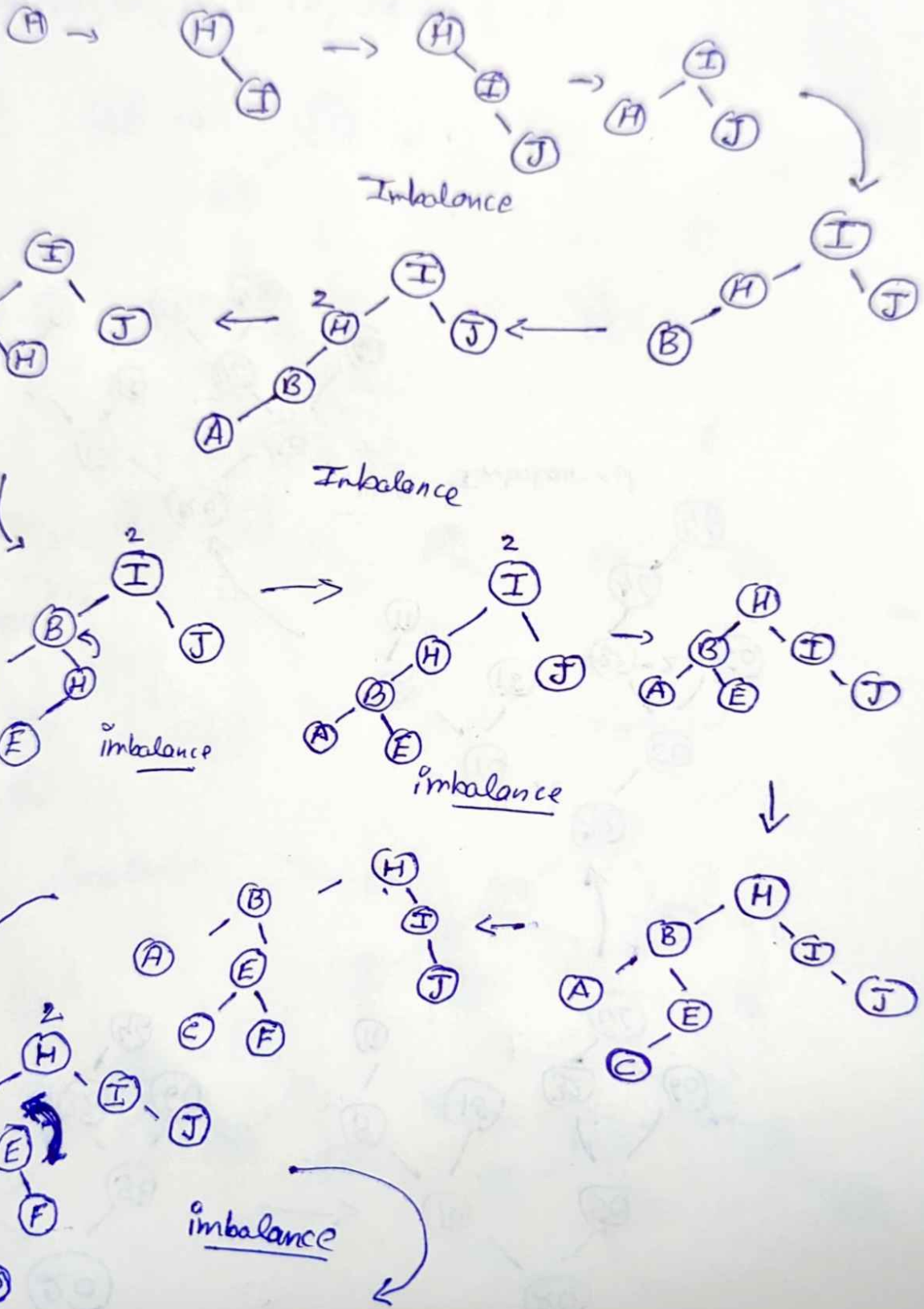


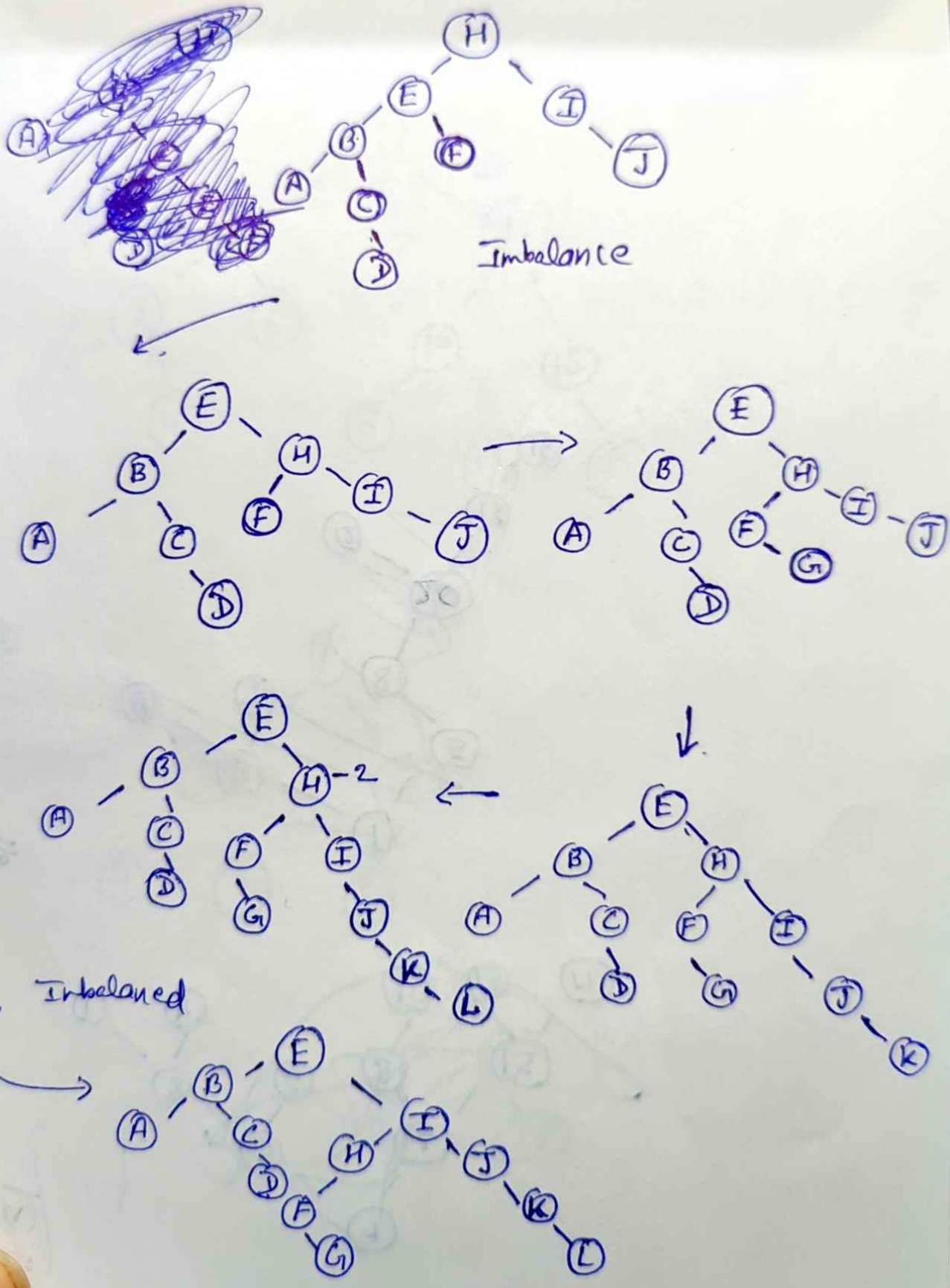





20
10
50
8
15
32
60
46
11
48
-2
Imbalanced.

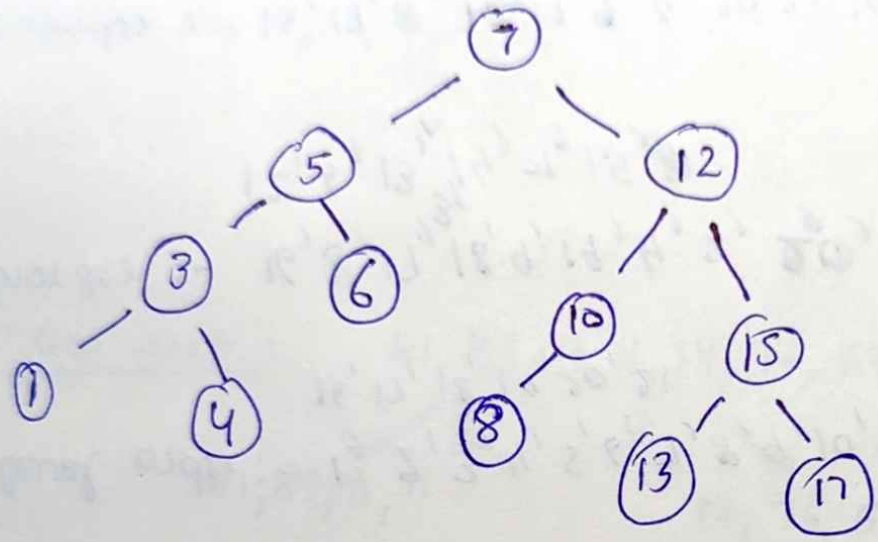
(b) H, I, J, B, A, E, C, F, D, G, K, L



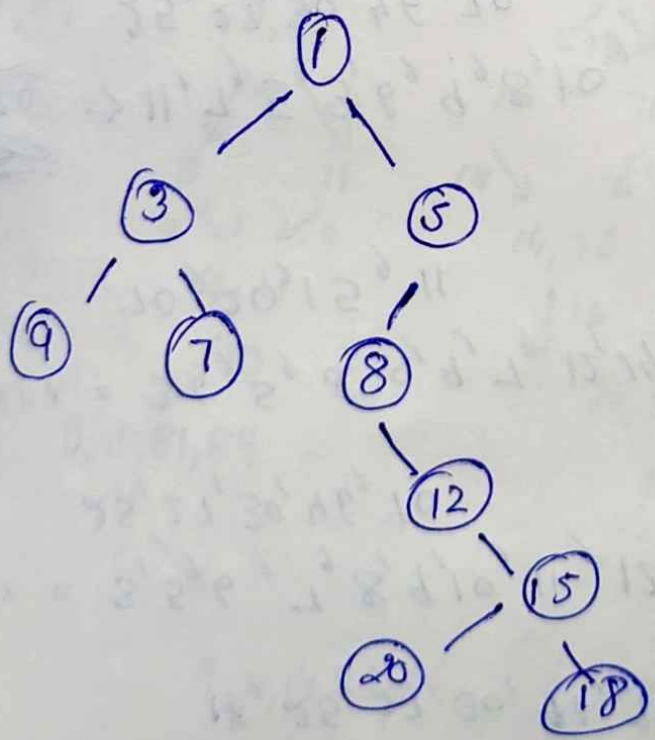

Imbalance
-2
Imbalanced

Q3

a



b



Q4

a

level order: 11, 7, 15, 5, 9, 13, 20, 3, 6, 8, 10, 12, 14, 18, 25, 27, 30, 46, 70

Inorder = 3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 18, 20, 25, 27, 30, 46, 70

postorder = 3, 6, 5, 8, 10, 9, 7, 12, 14, 13, 18, 25, 27, 30, 46, 70, 20, 15, 11

~~postorder~~

preorder => 11, 7, 5, 3, 6, 9, 8, 10, 15, 13, 12, 14, 20, 18, 25, 27, 30, 46, 70

b

level order :- 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21

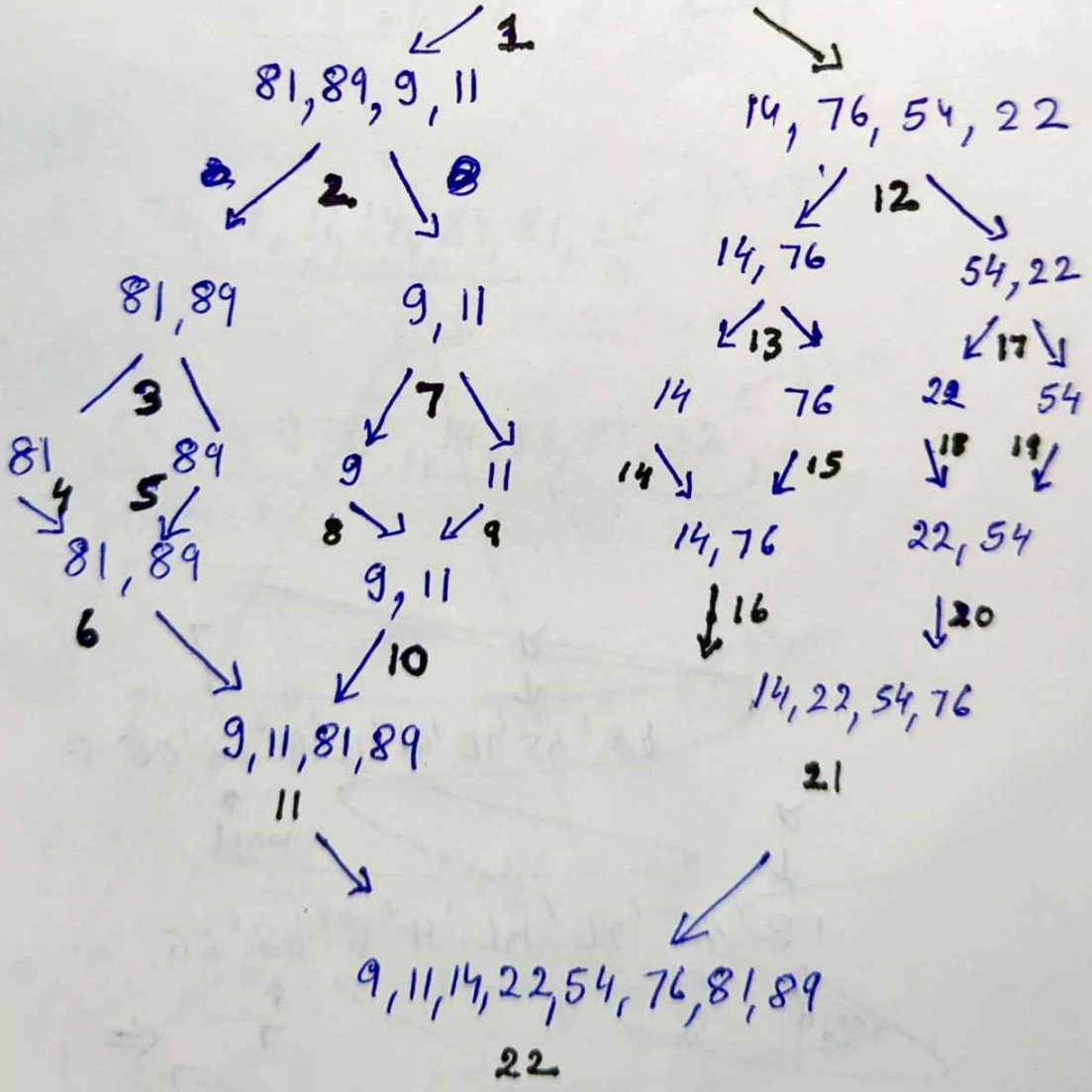
Inorder :- 16, 8, 17, 18, 9, 19, 4, 2, 20, 10, 21, 5, 11, 1, 12, 6, 13, 3, 14, 7, 15

postorder :- 16, 17, 8, 18, 19, 9, 4, 20, 21, 10, 11, 5, 2, 12, 13, 6, 14, 15, 7, 3, 1

preorder: 1, 2, 4, 8, 16, 17, 9, 18, 19, 5, 10, 20, 21, 11, 3, 6, 12, 13, 7, 14, 15

5

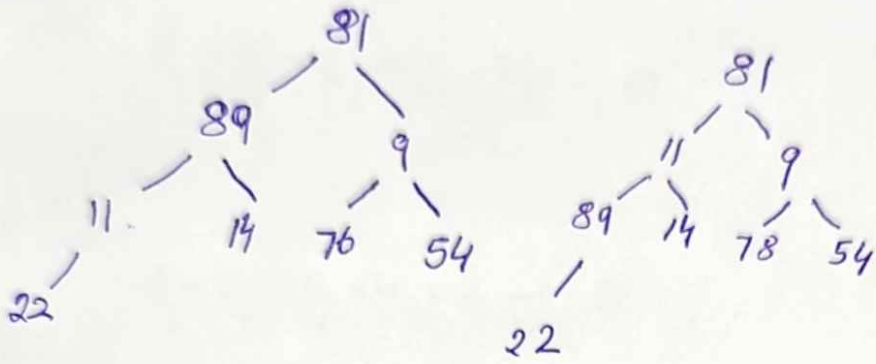
merge sort: 81, 89, 9, 11, 14, 76, 54, 22

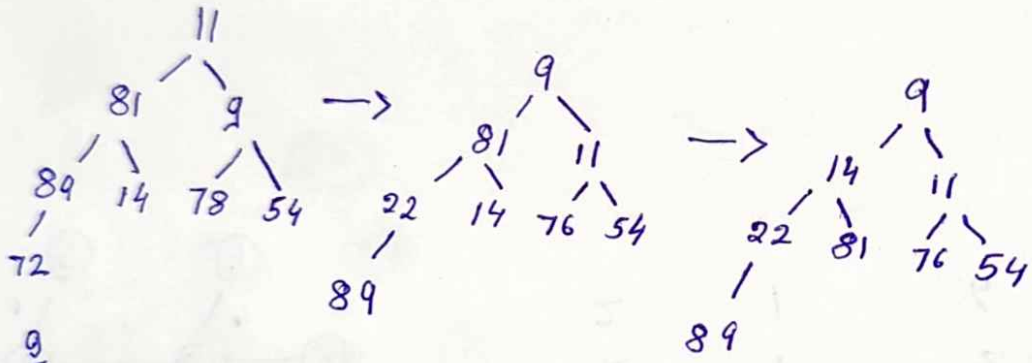


Heap Sort:

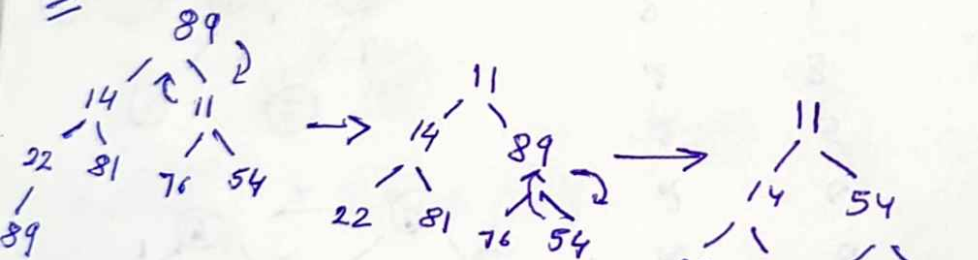
81, 89, 9, 11, 14, 78, 54, 22

Creating min heap:

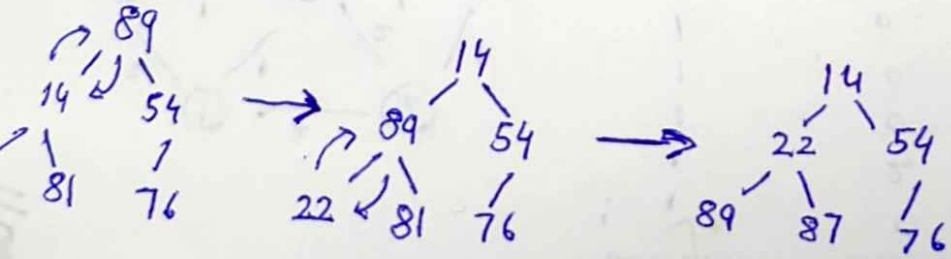




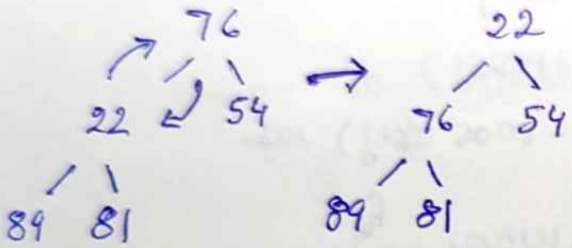
9



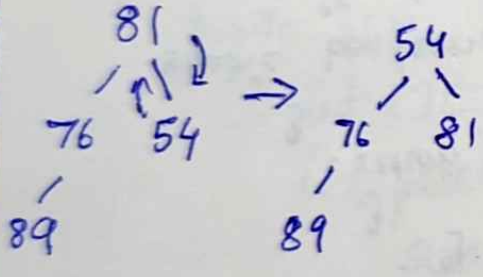
9, 11



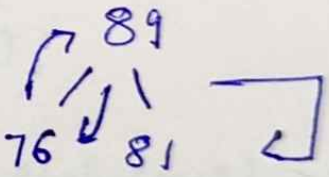
9, 11, 14

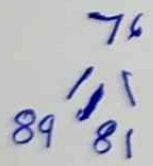


9, 11, 14, 22

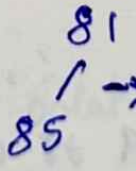


9, 11, 14, 22, 54





⇒ 9, 11, 14, 22, 54, 76



→ 9, 11, 14, 22, 54, 76, 81, 81

~~Heap sort~~

Q6

$-5, -8, -2, 0, -4, -1, -6$

$\Rightarrow$

step1

$-8, -5, -2, 0, -4, -1, -6$

step2 $\Rightarrow$ $-8, -5, -2, 0, -4, -1, -6$

step3 $\Rightarrow$ $-8, -5, -2, 0, -4, -1, -6$

step4 $\Rightarrow$ $-8, -5, -2, -4; 0, -1, -6$

~~step5 $\Rightarrow$ $-8, -5, -2, -4, -1, 0, -6$~~ step5 $-8, -5, -4, -2, 0, -1, -6$

~~step6 $\Rightarrow$ $-8, -5, -2, -4, -1, -6, 0$~~ step6 $-8, -5, -4, -2, -1, 0, -6$

step7

$-8, -5, -4, -2, -1, 0, -6$

step 8

$-8, -5, -4, -2, -1, -6, 0$

Step 9

$-8, -5, -4, -2, -6, -1, 0$

step 10

- -8, -5, -4, -6, -2, -1, 0

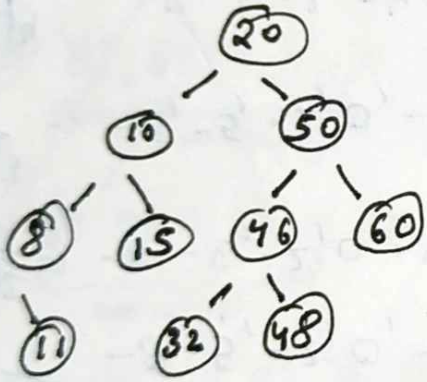
step 11

-8, -5, -6, -4, -2, -1, 0

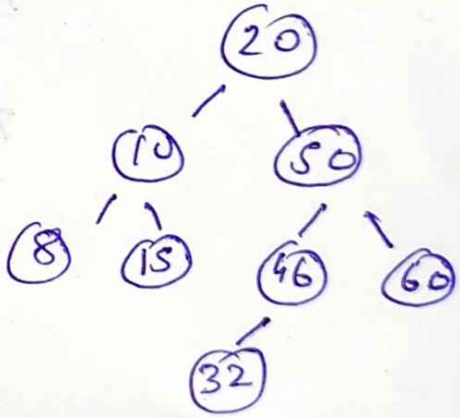
step 12

-8, -6, -5, -4, -2, -1, 0 .

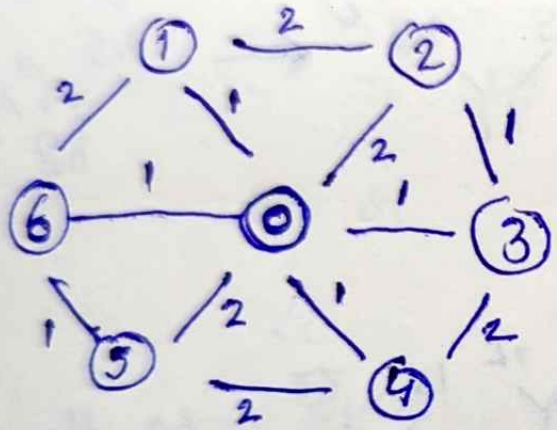
Q7

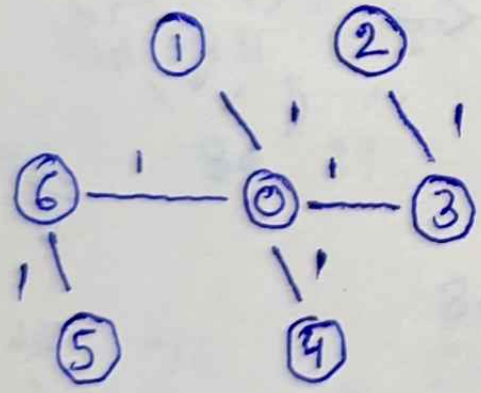


delete 48 & 11



8





weight	source	destination
1	0	1
1	2	3
1	0	3
1	0	4
1	0	6
1	6	5
2	0	2
2	0	5
2	5	4
2	3	4
2	1	2
2	1	6

Q9

Breadth first search :- A, B, C, D, E, F, G, H

Depth first search G, E, H, F, B, C, D, A

Q10

```
public class queenproblem {
    static boolean isafe (int [][] board, int row, int col){
        for (int i=0 ; i<col ; i++) {
            if (board [row][i] == 1) {
                return false;
            }
            if (row-i>=0 && board [row-i][col-i]==1){
                return false;
            }
            if (row+i< board.length && board [row+i][col-i]
                                                    ==1) {
                return false;
            }
        }
        return true;
    }
static boolean placequeen (int [][] board, int col) {
    if (col == board.length) {
        return true;
    }
for (int row = 0 ; row < board.length; row++) {
    if (issafe (board, row, col)) {
        board [row][col] = 1;
```

```
            if (placequeen(board, col+1)){
                return true;
            }
            board[row][col] = 0;
        }
    }
    return false;
}
public static void main(String[] args){
    int[][] board = new int[4][4];
    if(placequeen(board, 0)){
        System.out.println(Arrays.toString(row));
    }
} else {
    System.out.println("No solution");
}
}
}
```

Q1

```
class node {
    int data;
    Node left, right;
    Node (int data) {
        this.data = data;
        left = right = null;
    }
}
class binarysearchtree {
    Node root;
    boolean iterativeSearch (int key) {
        Node current = root;
        while (current != null) {
            if (current.data == key)
                return true;
            else if (current.data > key)
                current = current.left;
            else
                current = current.right;
        }
        return false;
    }
}
public class main {
    public static void main (String [] args) {
        BinarySearchTree bst = new BinarySearchTree ();
        bst.root = new Node (50);
```

```
        bst.root.left = new Node(30);
        bst.root.right = new Node(70);
        bst.root.left.left = new Node(20);
        bst.root.left.right = new Node(40);
        bst.root.right.left = New Node(60);
        bst.root.right.right = New Node(80);
        int key = 60;
        if (bst.iterativeSearch(key))
            System.out.println(key + );
        else
            System.out.println(key);
    }
}
```

Q2

```
class Node {
    int data;
    Node left, right;
    Node(int data) {
        this.data = data;
        left = right = null;
    }
}
class binarytree {
    Node root;
    void insert(int key) {
        root = InsertRec(root, key);
    }
```

```
Node insertRec (Node root, int key) {
    if (root == null) {
        root = new Node(key);
        return root;
    }
    if (key < root.data)
        root.left = insertRec(root.left, key);
    else if (key > root.data)
        root.right = insertRec(root.right, key);
    return root;
}
boolean search (int key) {
    return searchRec (root, key);
}
boolean searchRec (Node root, int key) {
    if (root == null)
        return false;
    if (root.data == key)
        return true;
    if (key < root.data)
        return searchRec (root.left, key);
    else
        return searchRec (root.right, key);
}
}
```

```
public class main {
    public static void main (String [] args) {
        Binary tree tree = new BinaryTree ();
        tree.insert (50);
        tree.insert (30);
        tree.insert (20);
        tree.insert (40);
        tree.insert (70);
        tree.insert (60);
        tree.insert (80);

        int key = 55;
        if (tree.search(key)) {
            System.out.println (key)
        else
            {
            tree.insert (key)
            System.out.println(key)    #not found.
            }
    }
}
}
```

03

```
class Node {
    int data;
    Node left, right;
    Node (int data) {
        this.data = data;
        left = right = null;
    }
}
```

```
class binarytree {
    Node root;
    void insert(int key) {
        root = insertrec(root, key);
    }
    Node insertrec(Node root, int key) {
        if (root == null) {
            root = new Node(key);
            return root;
        }
        if (key < root.data)
            root.left = insertRec(root.left, key);
        else if (key > root.data)
            root.right = insertRec(root.right, key);
        return root;
    }
    void selectionsort(int[] arr)
    { int n = arr.length;
      for (int i=0; i<n-1; i++) {
        int min_idx = i;
        for (int j = i+1; j<n; j++)
        { if (arr[j] < arr[min_idx])
            min_idx=j;
        }
        int temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }
}
```

```
public class main {
    public static void main (String [] args)
    { BinaryTree tree = new BinaryTree ();
      tree.insert (50);
      tree.insert (30);
      tree.insert (20);
      tree.insert (40);
      tree.insert (70);
      tree.insert (60);
    int [] arr = { 60, 20, 40, 30, 50, 70};
    tree.selectionSort (arr);
    System.out.println ("Sorted array");
    for (int i=0; i< arr.length; i++)
    { System.out.print (arr[i]);
    }
  }
}
```

04

```
public class Tower Of Hanoi {
    public static void main (String [] args) {
      int n=3;
      Tower Of Hanoi (n, 'A', 'C', 'B');
    }
```

```
public static void TowerOfHanoi(int n, char from_rod,
                                char to_rod, char
                                aux_rod) {
    if (n == 1) {
        System.out.println("Move disk 1 from rod " +
                from_rod + " to_rod" + to_rod);
        return;
    }
    towerofHanoi(n-1, from_rod, aux_rod, to_rod);
    System.out.println("Move disk" + n + "from rod" + from_rod
                + "to_rod" + to_rod);
    towerofHanoi(n-1, aux_rod, to_rod, from_rod);
}
}
```

05

```
public class reverse linked list {
    static class node {
        int data;
        Node next;
        Node (int data) {
            this.data = data;
            next = null;
        }
    }
```

```
static void reverseprint (Node head) {
    if (head == null) {
        return;
    }
    reverseprint (head.next);
    System.out.print (head.data + " ");
}
public static void main (String [] args) {
    Node head = new Node (1);
    head.next = new Node (2);
    head.next.next = new Node (3);
    head.next.next.next = new Node (4);
    System.out.print ("The original list: ");
    Node current = head;
    while (current != null) {
        System.out.print (current.data + " ");
        current = current.next;
    }
    System.out.print ("Reversed list ");
    reversePrint (head);
}
}
```